

Reinforcement Learning of Serpentine Locomotion for a Snake Robot

Ke Qiu, Hang Zhang, Yikai Lv, Yunkai Wang, Chunlin Zhou[†], Rong Xiong[†]

Abstract—The locomotion control of the snake robot is a challenging task that involves many degrees of freedom. Consider the rhythmic behaviour, we introduce the central pattern generator (CPG) model to provide serpentine locomotion, since it can reduce the control dimensionality while remaining continuous and flexible. However, the model parameters are usually chosen empirically, which is time consuming and complicated. Thus, we apply deep deterministic policy gradient (DDPG) algorithm of reinforcement learning (RL) to tune the model and obtain optimal parameters. Consequently, the hyper-redundant snake robot performs graceful serpentine locomotion with an RL controller in simulation. The flexibility of our CPG model and the feasibility of DDPG algorithm are verified according to the experimental results.

I. INTRODUCTION

The snake is born an experienced predator and an expert athlete. It is able to travel on forest floor, on desert and in water, exhibiting excellent mobility in various types of terrains. Inspired by the snake, snake robots are invented. They usually have many degrees of freedom, and move without the help of wheels or legs [11]. In 1972, Shigeo Hirose constructed the first snake robot called Active Cord Mechanism and gave a good overview [2]. In 2009, Staudahl Øyvind et al. provided a snake robot with a contact force sensing system that is able to realize obstacle-aided locomotion [3]. In 2012, Howie Choset et al. proposed the Unified Snake that is capable of traversing diverse environments [4]. The small cross section and serpentine body allows snake robots to take part in rescue tasks or inspection operations, especially in narrow and confined spaces, and hyper-redundant mechanism provides plenty of performance.

Many approaches to make the locomotion of snake robot are being developed in the meantime. In 1972, Hirose modelled a snake using wheeled links with no side slip and proposed a serpenoid curve to describe the gliding motion of a snake, which is the most common form of snake locomotion [3]. In 2012, Howie Choset et al. used a snake robot with sixteen degrees of freedom to present a variety of gaits over flat ground by using the virtual chassis method, that is, to treat the snake robot like a wheeled robot [5]. Motoyasu Tanaka focused mainly on the three-dimensional steering and stair climbing of snake robots with switching

constraints and passive wheels from 2014 to 2018 [6], [7], [8]. In recent years, reinforcement learning (RL) techniques are applied to robot locomotion control as a novel approach. In 2020, a model-free RL algorithm was introduced for solving target tracking tasks for a snake robot [9]. The RL based controller directly maps the visual inputs to joint positions.

As serpentine locomotion exhibits typical rhythmic patterns, the artificial central pattern generator (CPG) is naturally be adopted as a fundamental kinematic framework for the motion control of snake robots. It can perform periodic behaviours and maintain smoothness while parameters changing, as well as form distributed and coupled networks, thus being widely used for robot locomotion like the hexapod [10] or the fish [11]. In 2006, Alessandro Crespi et al. constructed an amphibious snake robot AmphiBot II controlled by an on-board CPG inspired by those found in vertebrates [12]. Zhelong Wang et al. applied a CPG model on a snake robot, adjusting the speed and direction by modulating the parameters [13] in 2016. Particularly, Ijspeert et al. used a robotic salamander and a spinal cord model to explain the ability of salamanders to switch between swimming and walking [14]. Then in 2019, deep deterministic policy gradient (DDPG) algorithm was used to this robotic salamander to smoothly move toward a desired target by adapting to the environmental change from the firm ground to the mud with the same spinal cord model [15].

This paper proposed to combine a CPG model and our snake robot as a basic kinematic framework for serpentine locomotion because of the efficiency of generating rhythmic behaviour and the similarity to the salamander robot. Then, we apply DDPG algorithm to learn to determine proper parameters for a certain goal, instead of directly mapping to joint angular positions. By this way, the motion can be more natural and the control simpler. Experimental results of the simulation validate our scenario.

The paper is organized as follows. In section II we combine a mathematical CPG model with our modular and hyper-redundant snake robot. In section III, the platform, observation space, action space and reward function are given to explain the learning procedure for rhythmic locomotion. The results of learning and the comparison of whether using an RL controller are presented in section IV. Conclusions are drawn in section V.

II. A KINEMATIC FRAMEWORK TO GENERATE SERPENTINE LOCOMOTION

In this section, we first make a brief description of CPG. Next, the structure of our snake robot is provided. Finally, a

Mr. Ke Qiu and Mr. Yikai Lv are with the School of Mechanical Engineering.

Mr. Hang Zhang is with the College of Electrical Engineering.

Mr. Yunkai Wang, Dr. Chunlin Zhou and Dr. Rong Xiong are with the State Key Laboratory of Industrial Control Technology and the College of Control Science and Engineering, Zhejiang University, Hangzhou, China.

[†]The corresponding e-mail address of Dr. Zhou is c.zhou@zju.edu.cn. The corresponding e-mail address of Dr. Xiong is rxiong@zju.edu.cn.

mathematical CPG model is given as a kinematic framework to generate serpentine locomotion.

A. Central Pattern Generator

CPG is a neural network capable of producing coordinated patterns of rhythmic activities, without any inputs from sensory feedback or higher-level centres [16]. It has been widely discovered in many creatures and form a fundamental architecture of rhythmic activities like swimming and walking [17], [18]. CPGs that are distributed in different segments can be also organized as a coupled network. For most creatures, rich types of locomotion are needed, and thus higher-level centres take responsibility for modulating CPGs in order to generate desired patterns. Under the guidance of neurobiology, CPG has been translated into mathematical model and has been increasingly used in the control of robot locomotion. Different types of neurobiological models include neuron models like [19] and oscillator models. The latter is on the basis of mathematical models of nonlinear differential equations such as phase oscillator [20], [21] and Matsuoka oscillator [22]. A spinal cord model has been constructed and applied to a salamander robot in order to switch gait between swimming and walking. See [14] for the implementation.

B. Structure of The Snake Robot

The snake robot contains eight independent modules and each module has two degrees of freedom. As seen in Fig.1, the components to constitute a module are two servo motors, several motor brackets, a shell and a connector. Four motor brackets fix two servo motors on the shell with screws. The shell in circular shape consists of two halves that wrap around servo motors. An aluminium connector provides mechanical connection between modules. If we set up a coordinate system and specify that positive x direction is along the spine of the snake robot and positive z direction points normally upwards, then the former servo motor controls the roll angle while the latter one controls the yaw angle. Servo motors are connected by a serial bus.

C. Central Pattern Generator Model of Our Snake Robot

On our snake robot, every joint is regarded as an oscillator and the output signal controls the joint angle. Inspired by the spinal cord model [14], we present ours with some modifications. Compared with the original model, we list the modifications below. (i) The spinal cord model uses a sine function as an interactive term in the differential equation set. We find in practice that an inverse tangent function works perfectly well due to its non-decreasing property. (ii) The spinal cord model uses a second order differential equation to make the amplitude approach to the target amplitude exponentially. Our model realizes the same thing but simpler. (iii) We use Fourier series as an output, thus able to generate any periodic signal theoretically.

Consider the i -th oscillator and the signal that we want this oscillator to produce is given. Without loss of generality, we assume that the periodic signal is an odd function, thus

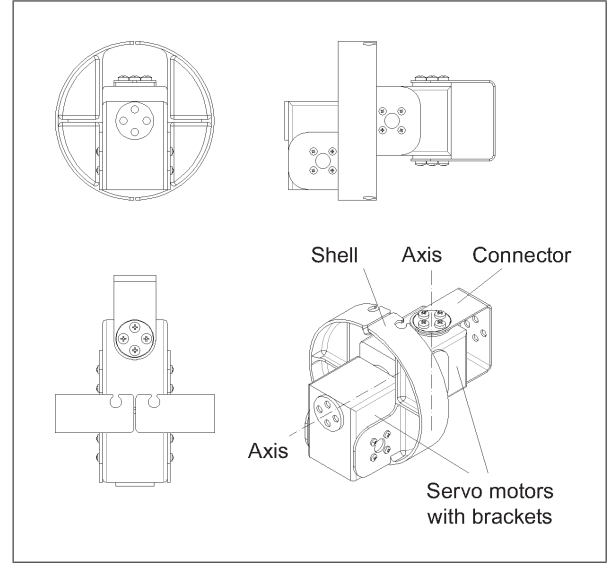


Fig. 1. A module assembly of the snake robot.

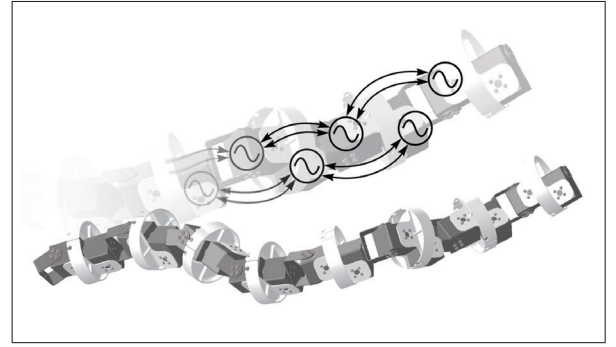


Fig. 2. The snake robot and its oscillator network. Joints having the same axis are coupled bidirectionally with nearest one.

can be expressed by Fourier series with only sine functions. Denote the Fourier coefficients by $R_{i0}, R_{i1}, R_{i2}, \dots, R_{in}$ if we expand the signal to order n . Then the system is given by:

$$\begin{aligned} \dot{\theta}_i &= \pi + \sum_j w_{ij} \arctan(\theta_j - \theta_i - \phi_{ij}), \\ \dot{r}_{ik} &= a(R_{ik} - r_{ik}) \quad (k = 0, 1, 2, \dots, n), \\ x_i &= \frac{r_{i0}}{2} + \sum_{k=1}^n r_{ik} \sin(k\theta_i), \end{aligned} \quad (1)$$

where (w_{ij}) is the weight matrix that determines the coupling network of all oscillators, (ϕ_{ij}) is the phase difference matrix that indicates θ_i lagging θ_j for ϕ_{ij} , and a is a positive time constant that represents the rate of exponential decay. The output of the oscillator is nothing but x_i , the desired signal.

Our CPG model in the snake robot include fifteen oscillators. Fig.2 show the construction of the oscillator network. The output of each oscillator is obtained by computing in Runge-Kutta method with order four, and sent as a command to the corresponding joint via the proportional-derivative controller. The problem is now simplified into finding the optimal Fourier coefficients for each oscillator and the proper

phase difference between oscillators.

However, for such a number of joints, choosing proper parameters is still an open question. It can be time consuming to determine them because this process would be tedious and extremely rely on trials or experiences. An RL method will be introduced in the next section, which gives suitable parameters through a policy.

III. A LEARNING POLICY FOR THE KINEMATIC FRAMEWORK

RL is a learning paradigm concerned with learning to control a system so as to maximize a numerical performance measure that expresses a long-term objective [23]. The basic idea is shown in Fig.3(a). The controller observes the state of the system and associated reward. It then calculates and sends an optimal action back to the system, making the system transfer to a new state and thus repeat the cycle. The rule maps a state to an action is called policy. Classical value iteration method works on problems with discrete and finite actions. The development of DDPG makes it extended to continuous and high-dimensional action spaces. DDPG is an algorithm that implement an actor-critic architecture via deep neural networks. As shown in Fig.3(b), the actor aims at improving the current policy while the critic evaluates the current policy by a Q-function. The algorithm then performs gradient ascent to reach the optimal parameters. We give a brief description of the algorithm that will be applied later. For more detailed explanation, please refer to [24].

In our task, the system is a robot platform that contains our snake robot and the plane on which the robot moves. The platform is constructed using PyBullet. We translate the design above into a URDF and create corresponding Gym environment for the snake robot. Observation space, action space and reward function play an important role in this environment. Denote the observation space by $\mathcal{X}$, action space by $\mathcal{A}$, reward at time t by R_t . Actor and critic are function approximations using neural networks with parameters η and μ , that is:

$$\pi_\eta : \mathcal{X} \rightarrow \mathcal{A}, \quad Q_\mu : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}. \quad (2)$$

The snake robot consists of 15 servo motors with feedback. Thus, angular positions of each joint can be coded and received. Also, we append the planar coordinates (i.e., displacements along x direction and y direction) to complete the

information. Hence, the observation space has 17 dimensions in total.

The action space is different than usual. Since we have put forward the CPG model, what controls the behaviour of the snake robot is nothing but CPG parameters, which are Fourier coefficients and phase differences. Fifteen oscillators make up our CPG model of the snake robot and we divide them into two sets. One set for controlling row angles, another for yaw angles. For simplicity we assign first order Fourier coefficient and a phase difference to each set. Hence, action space has an overall 4 dimensions.

Two tricks are employed during training. One is replay buffer $\mathcal{B}$, which stores previous transitions, and another is target networks, denoted by η_{tgt} and μ_{tgt} . The target is defined as:

$$Q_{tgt} = R_t + \gamma(1 - d)Q_{\mu_{tgt}}(X_{t+1}, \pi_{\eta_{tgt}}(X_{t+1})), \quad (3)$$

where d is a Boolean value that indicates if X_{t+1} is a terminal, $\gamma \in [0, 1]$ is a discount factor. The algorithm randomly selects a batch of transitions $(X_t, A_t, X_{t+1}, R_t, d) \in B \subseteq \mathcal{B}$, then take one step of gradient descent by

$$\nabla_\mu \frac{1}{|B|} \sum_B (Q_\mu(X_t, A_t) - Q_{tgt})^2 \quad (4)$$

and then gradient ascent by

$$\nabla_\eta \frac{1}{|B|} \sum_B Q_\mu(X_t, \pi_\eta(X_t)). \quad (5)$$

Finally, it updates the target network using polyak average:

$$\begin{aligned} \mu_{tgt} &= \tau\mu + (1 - \tau)\mu_{tgt}, \\ \eta_{tgt} &= \tau\eta + (1 - \tau)\eta_{tgt}. \end{aligned} \quad (6)$$

where $\tau \in [0, 1]$ is a factor usually close to 0. Moreover, a Gaussian noise is added to actions while exploring. In order to train a gait that is able to propel the snake robot towards the positive x direction, we have to carefully define the reward function. It contains two terms, so we are going to explain them respectively.

Continuous reward R_c evaluates the efficiency during an entire episode. For each step, find the deviations with respect to the x direction and the y direction, and define

$$R_c = \Delta x - |\Delta y|. \quad (7)$$

The function given by (7) implies how straightforward the snake robot moves to the target.

Discrete reward evaluates the final result that the snake robot achieves in one episode. Thus,

$$R_d = \begin{cases} 1, & \text{if } x \geq 1, \\ -1, & \text{if } x \leq -1 \text{ or } |y| \geq 1, \\ 0, & \text{otherwise.} \end{cases} \quad (8)$$

Obviously, if the snake robot reaches the target lying at $x = 1$, it will gain a reward, if it is far away from the target, it will gain a penalty. By the way, note that the environment would be reset when the snake robot either overstep the boundary or run out of maximum episode steps.

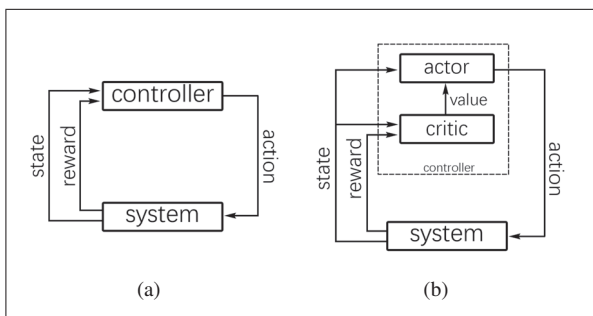


Fig. 3. Basic scenarios of RL (a) and DDPG (b).

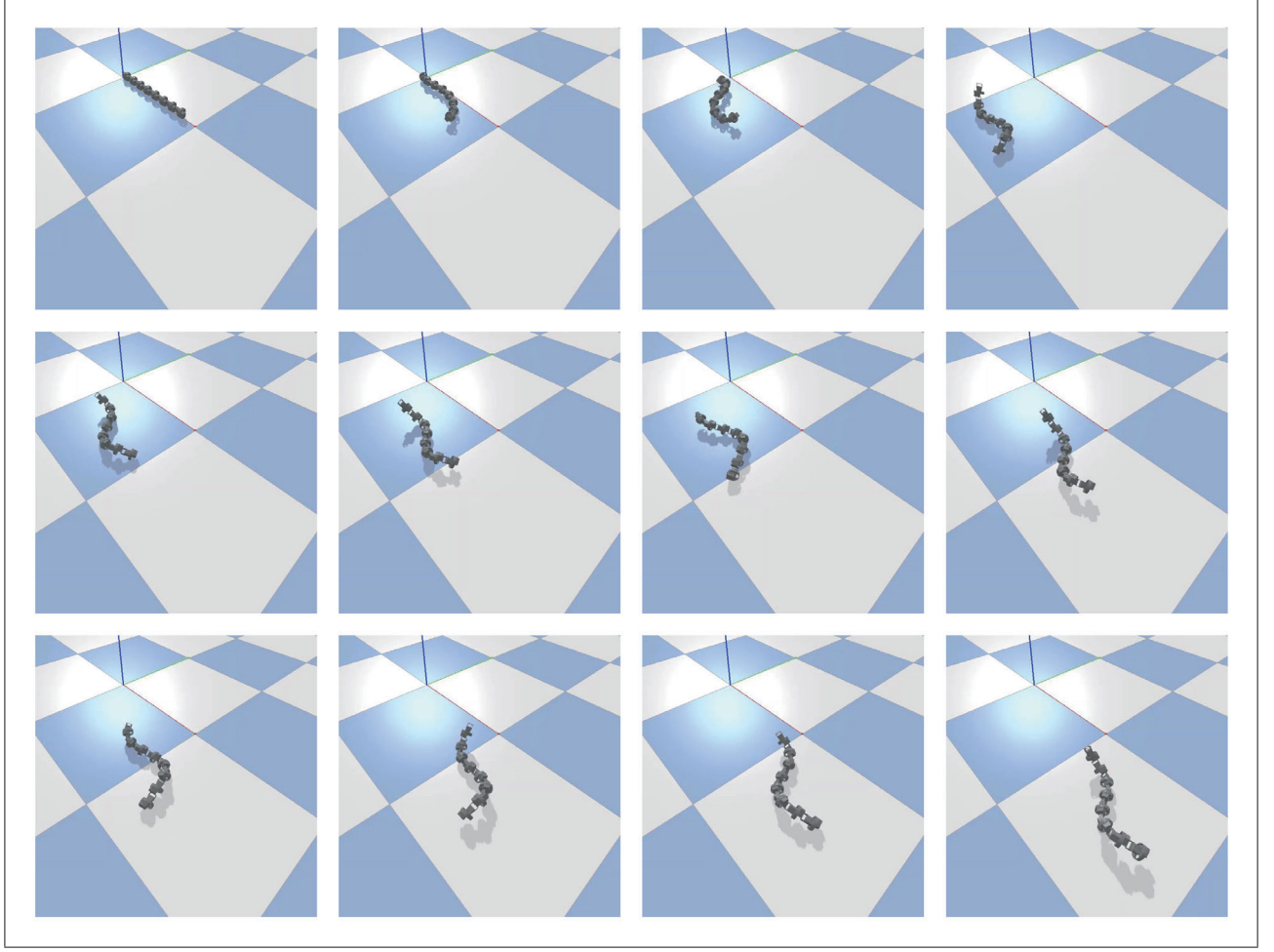


Fig. 4. Sequential snapshots during one episode. From left to right and top to bottom, snapshots are taken in every 1.5 seconds.

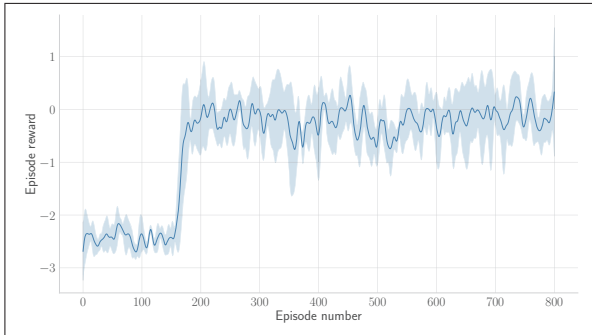


Fig. 5. Episode reward curve.

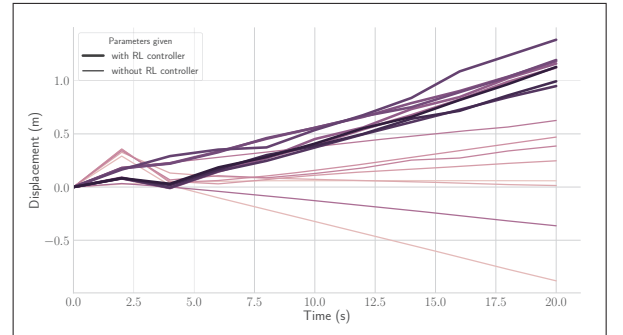


Fig. 6. Comparison of the results with or without an RL controller. Every colour stands for one experiment.

Now we can combine (7) and (8) to get our reward function:

$$R = w_c R_c + w_d R_d \quad (9)$$

where w_c and w_d are constant weights in training. This reward function promotes the snake robot moving forward with appropriate CPG parameters, as we will show in the following section.

IV. EXPERIMENTAL RESULTS

The learning procedures are carried on an Intel Core i5-8250U processor. Each time it is trained for 10,000 steps and at most 20 steps per episode. Five trainings take place independently by different random seeds. In the beginning 3,000 steps, the behaviours, or state transitions precisely, are collected in the replay buffer. After that, at the end of every episode, the target networks are updated using the data stored in the replay buffer. Hence, the robot initially presents

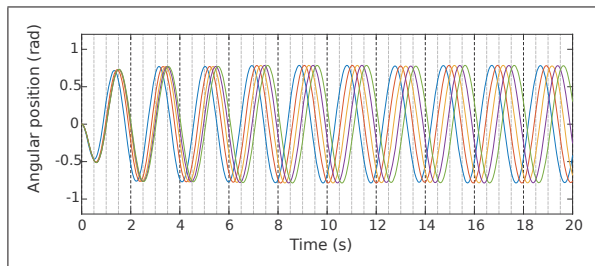


Fig. 7. Outputs of oscillators. Every colour represents one joint.

stochastic locomotion. In this period of time, transitions will be collected and stored. Once the training begins, the policy and Q-function are updated continuously, and thus the performance of the snake robot becomes more and more systematic.

The mean episode reward and the standard deviation are drawn with the line and the shadow respectively, in Fig.5. As we can see, the episode reward remains low at collecting stage, and it increases due to several training processes, and finally it reaches a relatively high level after only a few episodes. Although the result looks noisy because of the Gaussian noise adding to actions while exploring, the snake robot does find appropriate CPG parameters in the experiment.

To demonstrate, we make a comparison between two situations. In one situation, we assign empirical values to CPG parameters, which is obtained by trials or experiences. In another situation, we use the five RL based controllers that we have trained with different random seeds to determine these parameters. Every situation contains eight experiments. We evaluate the locomotion presented by the snake robot using the displacement along x direction. The result can be seen in Fig.6, with every colour standing for one experiment. In the former situation the performance of our snake robot is always excellent because the curves in bold are at the top. The observed gaits slightly vary from different RL controllers, but the evaluated mobilities are close. In the latter situation, it is of great uncertainty. Some parameters do help the robot move forward, some make it stay put, and some drive the snake backward. Although the dimension of the action space is not very large currently, the performance of the robot shows highly nonlinear property. To assign empirical values becomes much more complicated and trial and error becomes helpless especially when the dimension goes higher. However, RL based controller gives stable and efficient decisions, since all the eight curves in bold increase with an approximately constant velocity.

We still take sequential snapshots, as displayed in Fig.4. It can be seen that the snake robot shows a smooth and graceful posture by using our CPG model, whose parameters are chosen by the RL controller, moving along positive x direction. Fig.7 also records the angular position of some joints. Here, every colour represents one joint. As seen clearly, a stable phase difference is established. As a result, the flexibility of our CPG model and the feasibility of DDPG algorithm has been verified.

V. CONCLUSIONS

In this paper, we apply a CPG model to simplify the complex control problem. Then we find it difficult to determine the corresponding parameters of this model, considering such an amount of redundancy. To obtain suitable parameters, RL method is adopted to learn an optimal strategy that gives CPG parameters, under which our snake robot can perform graceful serpentine locomotion.

However, we are still looking for improvements. For instance, higher order Fourier coefficients can be considered to produce more sophisticated performance, as well as other types of network coupling. Furthermore, sensory feedback can be integrated into differential equations of our CPG model to help adjust output, which may realize self-tuning in varying terrain.

ACKNOWLEDGMENT

Financial supports by Zhejiang Provincial Natural Science Foundation of China (LY18F03002) and National Natural Science Foundation of China (61836015) are acknowledged.

REFERENCES

- [1] Transeth, Aksel & Pettersen, K.Y. & Liljebäck, Pål. (2009). A survey on snake robot modeling and locomotion. *Robotica*. 27. 999-1015. 10.1017/S0263574709005414.
- [2] Owen, Tony. (1994). Biologically Inspired Robots: SnakeLike Locomotors and Manipulators by Shigeo Hirose. *Robotica*. 12. 10.1017/S0263574700017264.
- [3] Liljebäck, Pål & Fjerdingen, Sigurd & Pettersen, K.Y. & Staudahl, Øyvind. (2009). A snake robot joint mechanism with a contact force measurement system. 3815-3820. 10.1109/ROBOT.2009.5152276.
- [4] Wright, Cornell & Buchan, Austin & Brown, Ben & Geist, Jason & Schwerin, Michael & Rollinson, David & Tesch, Matthew & Choset, and. (2012). Design and architecture of the unified modular snake robot. *Proceedings - IEEE International Conference on Robotics and Automation*. 10.1109/ICRA.2012.6225255.
- [5] Enner, Florian & Rollinson, David & Choset, Howie. (2012). Simplified motion modeling for snake robots. *Proceedings - IEEE International Conference on Robotics and Automation*. 4216-4221. 10.1109/ICRA.2012.6225163.
- [6] Tanaka, Motoyasu & Tanaka, Kazuo. (2015). Control of a Snake Robot for Ascending and Descending Steps. *Robotics, IEEE Transactions on*. 31. 511-520. 10.1109/TRO.2015.2400655.
- [7] Tanaka, Motoyasu & Nakajima, Mizuki & Suzuki, Yosuke & Tanaka, Kazuo. (2018). Development and Control of Articulated Mobile Robot for Climbing Steep Stairs. *IEEE/ASME Transactions on Mechatronics*. PP. 1-1. 10.1109/TMECH.2018.2792013.
- [8] Tanaka, Motoyasu & Tanaka, Kazuo & Matsuno, Fumitoshi. (2015). Approximate Path-Tracking Control of Snake Robot Joints With Switching Constraints. *Mechatronics, IEEE/ASME Transactions on*. 20. 1633-1641. 10.1109/TMECH.2014.2367657.
- [9] Bing, Zhenshan & Lemke, Christian & Morin, Fabric & Jiang, Zhuangyi & Huang, Kai & Knoll, Alois. (2020). Perception-Action Coupling Target Tracking Control for a Snake Robot via Reinforcement Learning. *Frontiers in Neurobotics*. 14. 10.3389/fnbot.2020.591128.
- [10] Chen, W., Ren, G., Zhang, J. et al. Smooth transition between different gaits of a hexapod robot via a central pattern generators algorithm. *J Intell Robot Syst* 67, 255-270 (2012).
- [11] Crespi, A., Lachat, D., Pasquier, A. et al. Controlling swimming and crawling in a fish robot using a central pattern generator. *Auton Robot* 25, 3-13 (2008).
- [12] Crespi, A., & Ijspeert, A. J. (2006). AmphiBot II: An amphibious snake robot that crawls and swims using a central pattern generator. In *Proceedings of the 9th international conference on climbing and walking robots (CLAWAR 2006)* (No. CONF, pp. 19-27).

- [13] Wang, Z., Gao, Q. & Zhao, H. CPG-Inspired Locomotion Control for a Snake Robot Basing on Nonlinear Oscillators. *J Intell Robot Syst* 85, 209–227 (2017).
- [14] Ijspeert, A.J. & Crespi, Alessandro & Ryczko, Dimitri & Cabelguen, Jean-Marie. (2007). From Swimming to Walking with a Salamander Robot Driven by a Spinal Cord Model. *Science* (New York, N.Y.). 315. 1416–20. 10.1126/science.1138353.
- [15] Cho, Younggil & Manzoor, Sajjad & Choi, Youngjin. (2019). Adaptation to environmental change using reinforcement learning for robotic salamander. *Intelligent Service Robotics*. 12. 10.1007/s11370-019-00279-6.
- [16] Ijspeert, A.J.. (2008). Central pattern generators for locomotion control in animals and robots: a review. *Neural Networks*. 21.
- [17] Delcomyn, F. (1980). Neural Basis of Rhythmic Behavior in Animals. *Science* (New York, N.Y.). 210. 492–8. 10.1126/science.7423199.
- [18] Stein, P.S. & Smith, J.L.. (1997). Neural and biomechanical control strategies for different forms of vertebrate hindlimb motor tasks. *Neurons, Networks, and Motor Behavior*. 61–73.
- [19] Kotaleski, Jeanette & Grillner, Sten & Lansner, Anders. (1992). Computer simulation of the segmental neural network generating locomotion in lamprey by using populations of network interneurons. *Biological cybernetics*. 68. 1–13. 10.1007/BF00203132.
- [20] Buchli, Jonas & Ijspeert, A.J.. (2004). Distributed Central Pattern Generator Model for Robotics Application Based on Phase Sensitivity Analysis. 3141. 333–349.
- [21] Cohen, Avis & Holmes, Philip & Rand, Richard. (1982). The nature of the coupling between segmental oscillators of the lamprey spinal generator for locomotion: A mathematical model. *Journal of mathematical biology*. 13. 345–69. 10.1007/BF00276069.
- [22] Matsuoka, Kiyotoshi. (1985). Sustained oscillations generated by mutually inhibiting neurons with adaptation. *Biological cybernetics*. 52. 367–76. 10.1007/BF00449593.
- [23] Szepesvári, Csaba. (2010). Algorithms for Reinforcement Learning. 10.2200/S00268ED1V01Y201005AIM009.
- [24] Lillicrap, Timothy & Hunt, Jonathan & Pritzel, Alexander & Heess, Nicolas & Erez, Tom & Tassa, Yuval & Silver, David & Wierstra, Daan. (2015). Continuous control with deep reinforcement learning. *CoRR*.